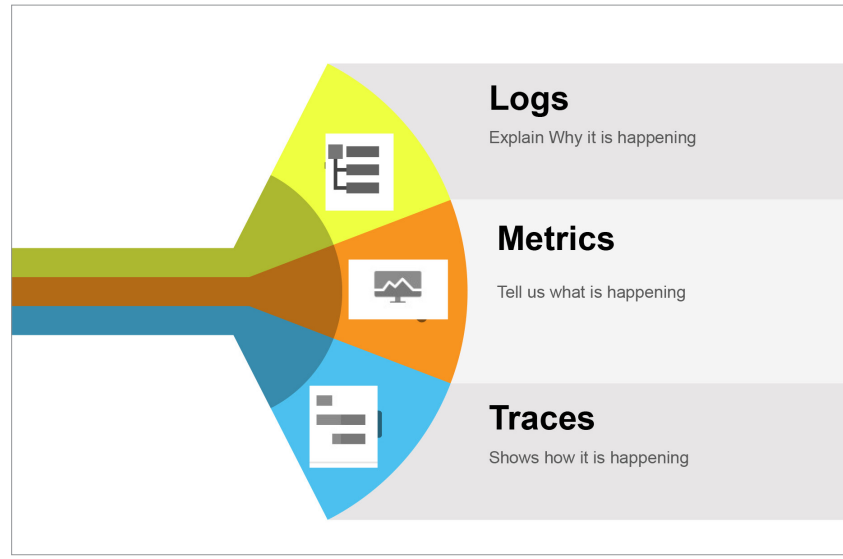
challenge by helping organisations understand not only what happened, but also why it happened. It uses metrics, logs, traces and other data sources to diagnose system issues beyond basic monitoring.

This has become increasingly important as enterprises move from single-provider environments to multi-cloud architectures. A single application request may now pass through multiple platforms, such as Azure for authentication, AWS for computing and Google Cloud for data operations. Observability tools provide visibility across this entire flow, helping organisations identify performance issues, track system behaviour and improve reliability.



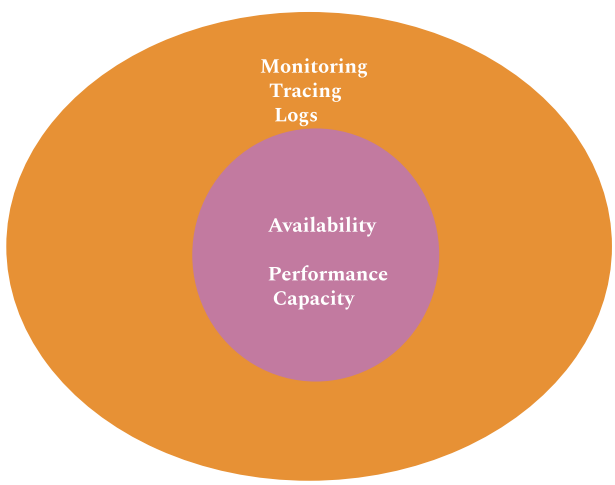
The three pillars of observability

The three pillars of observability: Logs, metrics and traces

- **Logs** capture information from system transactions and events in a running environment, including errors, warnings, notifications and other activities. Analysing logs helps identify why a system is not behaving as expected and determines the cause of failures.
- **Metrics** are KPI-based measurement parameters used to create dashboards and visualise performance indicators such as CPU utilisation, disk utilisation, network throughput, incoming and outgoing traffic, etc.
- **Tracing** records and tracks how a request moves end-to-end across different components of a system, providing visibility into the complete request flow.

Monitoring primarily focuses on system availability, performance and capacity, while observability extends beyond that by combining logs, traces and metrics to provide deeper insights into system behaviour. Monitoring

Monitoring v Observability



Aspect	Monitoring	Observability
Primary Goals	Identify and alert on known issues and metrics (When & What)	Understand and explore unknown and emergent behaviors (Why & How)
Scope	Primarily focused on uptime and system metrics	Encompasses logs, metrics, traces, and more to provide a holistic view
Data Sources	Metrics (e.g., CPU usage, memory, response time)	Metrics, logs, traces, events, and all output data sources
Approach	Pre-configured dashboards and alerting rules	Supports ad hoc querying and real-time analysis
Nature	Reactive	Proactive
Complexity	Easier to implement with standard metrics and thresholds	More complex, requires a deep integration for comprehensive data capture
Example Tools	Nagios, Zabbix, Prometheus (metrics-focused)	Grafana, OpenTelemetry, Jaeger
Use Case	Operators and IT teams monitoring uptime and performance	Investigate why response times have increased without a predefined alert

The major differences between monitoring and observability